And that's why we need to move away from inheritance in our designs. Instead, we need to be using *composition* wherever we see a case of A *uses a* B, or A *has a* B. Our persisted `Person` object won't subclass `DataWrapper`. Instead, it'll construct a reference to a database wrapper object and use that object reference to save and restore itself.

But that can also make code messy. And that's where a combination of mixins come to the rescue, because we can say this:

```
class Person
  include Persistable
  # ...
end
```

instead of this:

```
class Person < DataWrapper
  # ...
end
```

If you're new to object-oriented programming, this discussion may feel remote and abstract. But as you start to code larger and larger programs, we urge you to think about the issues discussed here. Try to reserve inheritance for the times where it is justified. And try to explore all the cool ways that mixins let you write decoupled, flexible code.

What's Next

In this chapter, we looked at using Ruby modules to encapsulate code into namespaces and to share code by using the `include` method to mix modules into classes. We also talked about how module inclusion affects method lookup and when to use mixins versus inheritance.

Now that we've learned some of Ruby's class and object structure, let's look at some of the classes that Ruby uses for standard types.